Python Engineering

WEEK 02 - Data Structures | CLASS 08 - Data Processing | EX16

Official class change: class_07_numpy_arrays is complete. We remain in week_02 and enter **class_08_data_processing**.

EX16 - Trajectory processing with masks and statistics

Now that arrays are familiar, the next step is using them to answer questions about data: which samples belong to ascent, which are above a threshold, what happens inside a time window, and which statistics summarize the trajectory.

1. Start from trajectory_array

```
# Convention from EX15
# columns: [h, v, t]

heights = trajectory_array[:, 0]
velocities = trajectory_array[:, 1]
times = trajectory_array[:, 2]
```

2. Boolean masks

```
ascending_mask = velocities > 0
descending_mask = velocities < 0
above_100_mask = heights > 100
```

A boolean mask is a True/False array aligned with the data. NumPy can use it to select only rows where the condition is True.

```
ascending_states = trajectory_array[ascending_mask]
above_100_states = trajectory_array[above_100_mask]
```

3. Combine conditions

```
high_and_ascending = (
    (heights > 50)
    & (velocities > 0)
)

selected = trajectory_array[high_and_ascending]
```

With NumPy arrays use &, |, and ~ rather than scalar and/or/not.

4. Time-window selection

```
time_window = (times >= 2.0) & (times <= 5.0)
window_states = trajectory_array[time_window]
```

5. Statistics

```
mean_height = np.mean(heights)
std_height = np.std(heights)
min_velocity = np.min(velocities)
max_velocity = np.max(velocities)
```

6. np.where()

```
indices_above_100 = np.where(heights > 100)[0]
```

`np.where(condition)` can recover the positions where a condition is satisfied.

7. Add a derived column

```
energy = (
    config['g'] * heights
```

```
    + 0.5 * velocities**2
)

processed_data = np.column_stack((
    heights,
    velocities,
    times,
    energy
))
```

processed_data now has four columns: `[h, v, t, energy]`. Generating derived variables from raw data is a common processing pattern.

8. Differences between samples

```
dt_samples = np.diff(times)
dh_samples = np.diff(heights)
```

`np.diff()` computes consecutive differences: x[i+1] - x[i]. The result has one fewer element than the input.

9. Your mission

```
Create: ex16_trajectory_data_processing.py

1. Reuse the EX15 trajectory_array.
2. Extract h, v, t columns.
3. Create ascent and descent masks.
4. Count ascent/descent states with np.sum(mask).
5. Select states with h > 100 m.
6. Select states between t = 2 s and t = 5 s.
7. Create a combined mask: h > 50 AND v > 0.
8. Compute mean/std of height and mean/std of velocity.
9. Use np.where() to get indices with h > 100 m.
10. Recompute specific energy and add it as a fourth column.
11. Use np.diff(times) and verify dt is approximately constant.
12. Print a compact engineering summary.
```

10. Questions you should answer

```
- How many samples belong to ascent?
- How many belong to descent?
- How many samples are above 100 m?
- Which indices are above 100 m?
- Mean and standard deviation of altitude?
- Is dt constant according to np.diff(times)?
- What is processed_data.shape after adding energy?
```

11. Completion criterion

EX16 is complete when you can translate an engineering question into a boolean condition, use that condition to filter an array, and obtain a numerical answer without unnecessary loops.